Python unter der Haube

Speichermodell, **is** vs. == und try/finally – für Java-Umsteiger

1. Alles ist ein Objekt

- In Python gibt es **keine primitiven Datentypen** wie `int` oder `double` in Java.
- Auch Zahlen, Wahrheitswerte und Strings sind **Objekte im Heap**.
- Python-`int` entspricht also Java-`Integer`, nicht Java-`int`. (Dafür ist es unbegrenzt groß – kein Überlauf!)

2. Variablen sind Namensschilder

- Eine Variable **enthält** kein Objekt, sie **zeigt** auf eines (Referenz).
- `b = a` kopiert nichts. Danach zeigen beide Namen auf **dasselbe** Objekt.
- `id(x)` liefert eine eindeutige Nummer des Objekts (in CPython: die Speicheradresse).

3. Veränderlich vs. unveränderlich

unveränderlich (immutable)	veränderlich (mutable)
`int, float, bool, str, tuple, None`	`list, dict, set`, eigene Klassen
kann nicht geändert, nur **ersetzt** werden	Änderungen sind über **alle** Namen sichtbar

```
a = 5
b = a
b += 1        # b zeigt auf ein NEUES Objekt 6, a bleibt 5

x = [5]
y = x
y.append(6)   # dasselbe Objekt wird geändert: x ist auch [5, 6]
```

Falle: += verhält sich je nach Typ unterschiedlich! Bei `int` entsteht ein neues Objekt, bei `list` wird das bestehende Objekt verändert. `a = [1]; b = a; b += [2]` → auch a ist jetzt `[1, 2]`.

4. Wiederverwendung (wie der Integer-Cache in Java)

Weil unveränderliche Objekte nicht verändert werden können, darf Python sie gefahrlos mehrfach verwenden. Das spart Speicher und Zeit.

- `True`, `False` und `None` gibt es jeweils nur ein einziges Mal.
- Kleine Ganzzahlen werden wiederverwendet: in CPython **-5 bis 256** (in Java: -128 bis 127).
- Kurze, einfache Strings wie Variablennamen werden ebenfalls wiederverwendet (Interning, in Java: String-Pool).

Wie weit das geht, hängt von der Python-Version und vom Kontext ab. Darauf darf man sich nicht verlassen!

5. Experimente: Wann ist a is b wahr?

Experiment A – die Schleife

```
for i in range(512):
    a = i
    b = i
    print(i, a is b)
```

Ergebnis: **immer True**, auch für 300 oder 511. Das beweist aber **nichts** über den Cache! a und b bekommen beide das Objekt, auf das `i` gerade zeigt – es gibt nur ein einziges Objekt. Das ist Regel 2 (Namensschilder), nicht Regel 4.

Experiment B – zwei getrennt erzeugte Objekte

```
a = int("256");  b = int("256");  a is b    # True  -> aus dem Cache
a = int("257");  b = int("257");  a is b    # False -> zwei neue Objekte
```

Experiment C – interaktiv vs. Skript

```
>>> a = 511          # interaktive Konsole: jede Zeile wird
>>> b = 511          # einzeln übersetzt
>>> a is b
False
```

Schreibt man dieselben drei Zeilen in eine Datei und startet sie mit `python3 skript.py`, kommt **True** heraus. Grund: Die ganze Datei wird auf einmal in Bytecode übersetzt, und der Compiler legt die Konstante 511 nur **einmal** an. In der Konsole ist jede Eingabe eine eigene Übersetzungseinheit – also zwei Objekte.

> **Fazit:** Ob `is` bei Zahlen oder Strings `True` liefert, ist ein Implementierungsdetail. Derselbe Code kann je nach Version und Ausführungsart unterschiedliche Ergebnisse liefern.

6. Die wichtigste Regel daraus

Python	Java	Bedeutung
`==`	`.equals()`	gleicher **Wert**
`is`	== (bei Objekten)	dasselbe **Objekt**

- **Werte immer mit `==` vergleichen.**
- `is` nur für `None` verwenden: `if x is None:`

```
a = int("1000")
b = int("1000")
a == b   # True:  gleicher Wert
a is b   # False: zwei verschiedene Objekte
```

7. Bytecode und try / finally

Python übersetzt den Quellcode zuerst in **Bytecode** (ähnlich wie Java in `.class`-Dateien). Diesen führt die Python-VM aus. Sie arbeitet den Code aber nicht stur „Zeile für Zeile“ ab: Der Bytecode enthält **Sprungbefehle** und eine **Exception Table**, die der VM genau sagt, wo der `finally`-Block liegt.

```
def verwirrende_funktion():
    try:
        print("1. Ich bin im try-Block.")
        return "Wert aus try"
    except ZeroDivisionError:
        return "Wert aus except"
    finally:
        print("2. Ich bin im finally-Block.")
        return "Wert aus finally (Ich gewinne!)"

ergebnis = verwirrende_funktion()
print(f"Ergebnis: {ergebnis}")
```

Ausgabe:

```
1. Ich bin im try-Block.
2. Ich bin im finally-Block.
Ergebnis: Wert aus finally (Ich gewinne!)
```

Was passiert?

- Bei `return "Wert aus try"` wird der Rückgabewert schon berechnet und **zwischengespeichert** – die Funktion ist aber noch nicht verlassen.
- Vorher **muss** der `finally`-Block laufen. Das ist garantiert, egal ob `return`, `break` oder eine Exception.
- Das `return` im `finally` **überschreibt** den zwischengespeicherten Wert. Würde eine Exception fliegen, würde sie sogar **verschluckt**.
- Java verhält sich hier übrigens genauso.

Selbst nachsehen: Mit dem Modul `dis` kann man den Bytecode anzeigen:

```
import dis
dis.dis(verwirrende_funktion)
# Am Ende steht die "ExceptionTable": Sie sagt der VM,
# zu welcher Stelle sie bei einer Ausnahme springen muss.
```

Guter Stil: Kein `return` im `finally`! `finally` ist zum **Aufräumen** da (Datei schließen, Verbindung trennen). Ab Python 3.14 gibt der Compiler dafür sogar eine `SyntaxWarning` aus (PEP 765). Noch besser für Aufräumarbeiten: `with`.

8. Übungen: Was wird ausgegeben?

```
# Aufgabe 1
s = "Hallo"
t = s
t += " Welt"
print(s)

# Aufgabe 2
l = [1, 2]
m = l
m += [3]
print(l)

# Aufgabe 3
def f(liste):
    liste.append(99)
    liste = [0]

x = [1]
f(x)
print(x)

# Aufgabe 4
x = None
print(x == None, x is None)   # Welche Schreibweise ist die richtige?

# Aufgabe 5
def g():
    for i in range(3):
        try:
            return i
        finally:
            print("finally", i)

print(g())
```

Lösungen

- **1:** `Hallo` – `str` ist unveränderlich, `t` zeigt danach auf ein neues Objekt.
- **2:** `[1, 2, 3]` – += verändert die Liste selbst.
- **3:** `[1, 99]` – append ändert das Objekt; `liste = [0]` biegt nur den lokalen Namen um.
- **4:** `True True` – richtig ist `x is None`.
- **5:** `finally 0`, dann `0` – `finally` läuft vor dem Verlassen der Funktion, die Schleife wird nicht fortgesetzt.